PCIe Diagnosis & Root-Cause Playbook

Localizing a PCIe Failure on Linux — Signal Integrity vs Protocol vs Power vs Thermal vs Firmware

May 2026

Contents

1 How to Use This Playbook

The Study Guide's PCIe chapter teaches PCIe at the register level as part of the whole job. This playbook is the bench artifact — the thing you keep open on a second monitor when a board is on the fixture throwing errors and you have to name the root cause before you can fix it. It is deliberately narrow: *given a PCIe link that is misbehaving, how do you localize the failure to a layer and a physical cause, with the exact commands, in a defensible order.*

The single idea behind everything here:

> A PCIe failure is not "errors=5." It is a *tuple*: which AER bits, on which lane, with which header log, at which retrain count, with how much eye margin, at what temperature and load. That tuple *is* the root cause. The whole job of this playbook is to collect that tuple efficiently and read it.

Conventions. BDF = `domain:bus:device.function`, e.g. `0000:03:00.0`. DSP = Downstream Port (root port or switch downstream port). USP = Upstream Port (an endpoint, or a switch's upstream port). EP = endpoint. Config offsets are bytes; bit numbering is LSB = 0. "GT/s" is the raw per-lane symbol rate; "Gen" is the speed code. Everything here is in-band Linux — no oscilloscope required to *localize* (you reach for the scope only to confirm a power/SI hypothesis, §10).

Speed reference (per lane, after encoding overhead). Knowing the usable throughput tells you whether a "downgraded" link still meets the data budget — sometimes a Gen4-trains-Gen3 fallback is harmless, sometimes it halves your NVMe bandwidth.

Gen	GT/s (per lane)	Encoding	Usable per lane	LnkSta CLS code
1	2.5	8b/10b	0.25 GB/s (250 MB/s)	1
2	5.0	8b/10b	0.50 GB/s (500 MB/s)	2
3	8.0	128b/130b	0.985 GB/s	3
4	16.0	128b/130b	1.969 GB/s	4
5	32.0	128b/130b	3.938 GB/s	5
6	64.0	PAM-4 + FLIT/FEC (1b/1b, 242B/256B)	7.563 GB/s	6

Gen1/2 lose 20% to 8b/10b; Gen3-5 lose only ~1.5% to 128b/130b (the big jump is here); Gen6 switches to PAM-4 signalling (2 bits per symbol, so 64 GT/s at the same ~32 GBd Nyquist as Gen5) plus a FLIT format with FEC and a strong CRC. Multiply the per-lane figure by the negotiated width for the link total (e.g. Gen4 x16 $\approx$ 31.5 GB/s usable each direction).

Safety / "don't break the line" notes are inline and flagged. `setpci` writes, `aer-inject`, and retrains can perturb a live link; do them on a dev station or with the DUT quiesced, never blindly on a production unit mid-soak.

2 The 30-Second Decision Tree

Before the detail, the shape of the whole method. Each numbered branch has its own chapter; the evidence column is what *confirms* that branch (not just what's consistent with it).

```
0. ARM. Clear AER (W1C). Record: LnkCap both ends, current speed/width,
   DPC state, ASPM/L1SS state, severity+mask, temperature. (sec 3)

1. Does it ENUMERATE?  (lspci -nn)
   NO  -> LTSSM stuck in Detect/Polling/Config. (sec 8)
          Cause set: power rails / REFCLK / PERST# / bifurcation / dead PHY / AC-cap.
          Evidence: dmesg "link training" stuck; rails on scope; BIOS bifurcation
          vs schematic; reseat changes it.

2. Right SPEED and WIDTH?  (current vs max; lspci prints "downgraded")
   SPEED low (Gen4->Gen3) -> EQUALIZATION / SI margin / BIOS gen-cap / THERMAL. (sec 9)
          Evidence: dmesg gen-change; retest hot AND cold; pcilmr margin shrinks;
          LnkCap ceilings agree but trained speed is lower.
   WIDTH low (x16->x8)    -> connector / dead lane / bifurcation / lane-reversal. (sec 8)
          Evidence: per-lane pcilmr finds the dead lane; reseat; BIOS bifurcation.

3. ERRORS under stress?  Decode WHICH AER bits. (sec 4)
   Correctable cluster {RxErr, BadTLP, BadDLLP, ReplayTO, RollOver}
          -> PHYSICAL / SIGNAL INTEGRITY. (sec 4,sec 9)
          Evidence: count rises with temperature; pcilmr margin low/one lane;
          reseat or cable swap changes it; a better TX preset improves it.
   Uncorrectable {CmplTO, UnexpCmpl, MalformedTLP, FCP, RxOverflow}
          -> PROTOCOL / FIRMWARE / UPSTREAM. (sec 4,sec 5,sec 7)
          Evidence: header-log decode names requester/type/address; reproducible
          regardless of temp/reseat; tied to a traffic pattern or FW/driver version.
   {PoisonedTLP, ECRC} -> DATA CORRUPTION upstream / through switch or retimer. (sec 12)
   {SurpriseDown}      -> POWER / mechanical. (sec 10)
          Evidence: correlates with load steps / rail droop on scope; reseat; PSU.

4. Errors ONLY hot or ONLY under load -> THERMAL SI or POWER droop. (sec 9,sec 10)
          Evidence: thermal chamber sweep; rail on scope under load; margin-vs-temp.

5. Nothing reproduces but field-flaky -> MARGIN is the discriminator. (sec 9)
          A pcilmr margin below limit on one lane is a latent SI fail a pass/fail
          link-up test misses entirely.
```

The reason this works: different root causes leave different evidence in different registers. SI shows up as correctable clusters that move with temperature and shrink the eye margin; protocol shows up as uncorrectable bits whose header log names a transaction; power shows up as Surprise Down / load-correlated bursts; firmware shows up as reproducible, temperature-independent, version-tied uncorrectables. The discriminator table in §14 is the whole playbook on one page.

3 Arm Before You Measure

PCIe AER status bits are write-1-to-clear (W1C) and they latch from boot — through power-on glitches, enumeration, link training, and every prior test. If you read them without arming first, you are reading history, not your stress window. Arm → stress → read is the only valid sequence, and "arm" is more than clearing one register.

3.1 The arm checklist

```
BDF=0000:03:00.0

# 1. Record the static facts BEFORE you touch anything (these don't change under test):
lspci -vvv -s $BDF | sed -n '/LnkCap:/p; /LnkSta:/p; /LnkCap2:/p; /LnkSta2:/p'
cat /sys/bus/pci/devices/$BDF/max_link_speed   /sys/bus/pci/devices/$BDF/current_link_speed
cat /sys/bus/pci/devices/$BDF/max_link_width   /sys/bus/pci/devices/$BDF/current_link_width

# 2. Record severity + mask FIRST so you know which bits will trigger DPC / link reset
#    UNDER you mid-test (severity/mask detail in sec 4). ECAP_AER offsets: +0x08 mask(unc),
#    +0x0C severity(unc), +0x14 mask(cor).
setpci -s $BDF ECAP_AER+0x0c.L     # uncorrectable severity (1=Fatal -> ERR_FATAL/DPC)
setpci -s $BDF ECAP_AER+0x08.L     # uncorrectable mask
setpci -s $BDF ECAP_AER+0x14.L     # correctable mask

# 3. Record DPC + ASPM/L1SS state (these change what a fatal error DOES to your test):
lspci -vvv -s $BDF | grep -A2 -iE 'DPC:|ASPM|L1SubCtl'

# 4. ARM: clear correctable + uncorrectable status (W1C). Write-back-the-set-bits is the
#    precise form; write-all-ones is the blunt form. Both clear; the precise form lets a
#    "won't clear" bit be a finding.
setpci -s $BDF ECAP_AER+0x10.L=0xffffffff   # clear COR_STATUS
setpci -s $BDF ECAP_AER+0x04.L=0xffffffff   # clear UNCOR_STATUS

# 5. VERIFY the arm took (both should now read 0):
setpci -s $BDF ECAP_AER+0x10.L ECAP_AER+0x04.L
```

If a status bit refuses to clear, that is itself a result: a hardware fault is re-latching it every cycle (a stuck PHY, a persistent internal error). Do not paper over it by re-clearing in a loop — record it.

3.2 Who else is clearing your latches

The kernel `pcieport`/AER driver attaches to root ports and RCECs and will read and clear AER status out from under you to print its `dmesg` lines. For deterministic manufacturing measurement you have three choices, in order of preference:

1. Read the kernel's own counters instead of racing it. With `CONFIG_PCIEAER_STATS`, the kernel exposes monotonic counters per device that *it* maintains:

   ```
   cat /sys/bus/pci/devices/$BDF/aer_dev_correctable   # named cor counters, kernel-maintained
   cat /sys/bus/pci/devices/$BDF/aer_dev_nonfatal
   cat /sys/bus/pci/devices/$BDF/aer_dev_fatal
   ```

   These are per-named-error tallies (RxErr, BadTLP, …) that survive between your polls and are not cleared by your `setpci`. For a station tool this is often the *robust* source — you delta them across the stress window instead of fighting the kernel for the W1C bits.

2. Own the registers in a window where the kernel won't report. Mask the correctable class you're counting, arm, stress, read raw status, then restore — accepting that the kernel won't log during your window.

3. Disable kernel AER for an A/B test (`pci=noaer` on the kernel cmdline) to prove the kernel path itself isn't the noise source. Diagnostic only — you don't ship with AER off.

   The reconciliation trap. If both your tool and the kernel clear the same W1C bits, each can hide errors from the other and your counts will be wrong and irreproducible. Pick one owner per run and *say which one* in the record. The cleanest station design reads `aer_dev_*` (kernel-owned) and never clears the raw bits at all.

4 AER in Depth — The Bits Are the Layer

AER (Advanced Error Reporting) is the extended capability (ID `0x0001`) where the hardware latches correctable and uncorrectable errors. The whole diagnostic value is that the bit names the layer, which names the root-cause class.

4.1 AER register map (offsets relative to the AER cap base)

Off	Register	Notes
`+0x00`	Capability Header	ID `0x0001`
`+0x04`	Uncorrectable Error Status (`UNCOR_STATUS`)	W1C. Non-zero after a clean run = FAIL
`+0x08`	Uncorrectable Error Mask	1 = not reported (may still latch in status)
`+0x0C`	Uncorrectable Error Severity	1 = Fatal (-> ERR_FATAL, link reset / DPC), 0 = Non-Fatal
`+0x10`	Correctable Error Status (`COR_STATUS`)	W1C. Recovered errors; rising count = SI concern
`+0x14`	Correctable Error Mask	1 = masked
`+0x18`	Adv. Error Cap & Control (`ERR_CAP`)	First Error Pointer **`[4:0]`** (which UNCOR bit the header log belongs to); ECRC gen/chk; FLIT-log bits
`+0x1C`	Header Log (16 B)	First 4 DWORDs of the TLP that caused the first uncorrectable error
`+0x2C`	Root Error Command	Root ports / RCEC only
`+0x30`	Root Error Status	Root ports only: which class(es) received, multi-error, first-fatal, AER IRQ msg #
`+0x34`	Error Source ID	Root ports only: requester ID of the COR and UNCOR source

> Root-port-only caveat (important and often gotten wrong). `Root Error Command` (+0x2C), `Root Error Status` (+0x30), and `Error Source ID` (+0x34) exist only on Root Ports and Root Complex Event Collectors, not on endpoints or switch downstream ports. On an endpoint those offsets are not the root-error registers. When you want "which requester caused this," read the Error Source ID on the root port above the device, not on the device — the root port is where the kernel AER driver assembles the OS-level story.

4.2 What it looks like in **`lspci -vvv`** (and how the names map)

`lspci` decodes the AER capability into named flags with + (set) / – (clear). This is the same data as the raw registers — here is the mapping you read off in seconds:

```
Capabilities: [100 v2] Advanced Error Reporting
    UESta:  DLP- SDES- TLP- FCP- CmpltTO+ CmpltAbrt- UnxCmplt- RxOF- MalfTLP- ECRC- UnsupReq- ACSViol-
    UEMsk:  DLP- SDES- TLP- FCP- CmpltTO- CmpltAbrt- UnxCmplt- RxOF- MalfTLP- ECRC- UnsupReq- ACSViol-
    UESvrt: DLP+ SDES+ TLP- FCP+ CmpltTO- CmpltAbrt- UnxCmplt- RxOF+ MalfTLP+ ECRC- UnsupReq- ACSViol-
     CESta:  RxErr+ BadTLP+ BadDLLP- Rollover- Timeout+ AdvNonFatalErr+
     CEMsk:  RxErr- BadTLP- BadDLLP- Rollover- Timeout- AdvNonFatalErr+
     AERCap: First Error Pointer: 0e, GenCap+ CGenEn- ChkCap+ ChkEn-
     HeaderLog: 00000001 00200a03 05010000 00050100
```

Mapping to the registers (and to the bit tables below):

`lspci` line	Register (offset)	Read it as
`UESta:`	UNCOR_STATUS (`+0x04`)	which uncorrectables LATCHED. Above: CmpltTO+ -> Completion Timeout fired.
`UEMsk:`	UNCOR_MASK (`+0x08`)	masked = not reported (may still latch in UESta).
`UESvrt:`	UNCOR_SEVERITY (`+0x0C`)	+ = Fatal (-> ERR_FATAL / DPC). Above CmpltTO is Non-Fatal here.
`CESta:`	COR_STATUS (`+0x10`)	which correctables latched. Above: RxErr+ BadTLP+ Timeout+ = the SI cluster.
`CEMsk:`	COR_MASK (`+0x14`)	masked correctables.
`AERCap:`	ERR_CAP (`+0x18`)	First Error Pointer: 0e = bit 14 = CmpltTO owns the HeaderLog; ECRC GenCap/ChkCap.
`HeaderLog:`	HEADER_LOG (`+0x1C`)	the 4 DWORDs of the offending TLP (decode in sec 5).

So this one block says: *the link is taking SI-class correctables (RxErr/BadTLP/ReplayTO, §9) AND a Completion Timeout latched whose header log is the MRd decoded in §5.* Two different stories in one capability — which is exactly why you decode the whole thing, not just "errors: yes."

> **`First Error Pointer`** is the glue. It is the UNCOR bit number (here `0x0e` = 14 = CmpltTO) that the HeaderLog belongs to. If you see a header log but read the wrong UNCOR bit as its cause, you chase the wrong TLP. Always pair `First Error Pointer` with the HeaderLog. (Newer FLIT-mode parts add a "TLP Prefix Log" and a "logged in FLIT mode" bit in this register; on a Gen6 link confirm the log is the TLP, not a prefix.)

4.3 Correctable bits (**`COR_STATUS`**, +0x10) — the SI early-warning system

Bit	Mask	Name	Meaning / typical root cause
0	`0x0001`	Receiver Error	Raw PHY symbol / 8b10b / 128b130b / sync-header error. Pure SI (loss, jitter, crosstalk, marginal EQ).
6	`0x0040`	Bad TLP	TLP with bad LCRC or bad sequence number -> NAK'd & replayed. SI (corruption on the wire).
7	`0x0080`	Bad DLLP	An ACK/NAK/FC/PM DLLP failed CRC. SI (DLLPs aren't retried, just dropped).
8	`0x0100`	REPLAY_NUM Rollover	Replay counter wrapped (4 consecutive replays of one TLP). SI / marginal link.
12	`0x1000`	Replay Timer Timeout	No ACK before REPLAY_TIMER expired -> replay. Classic marginal-link symptom.

Bit	Mask	Name	Meaning / typical root cause
13	`0x2000`	Advisory Non-Fatal	An uncorrectable error was *demoted* to advisory (e.g. an UnsupReq/CmplTO spec says report as correctable). Read the uncorrectable status to see what was demoted.
14	`0x4000`	Corrected Internal Error	Device-internal error the device corrected (its own RAM ECC). Device-side, not link.
15	`0x8000`	Header Log Overflow	More uncorrectables arrived than the 1-deep header log could hold. Means *many* uncorrectables – go fix those.

The single most important correctable signature: *Replay Timer Timeout* + *Bad TLP* + *REPLAY_NUM Rollover clustering* = the link is repeatedly corrupting TLPs and retrying = physical-layer / signal integrity. If you see this cluster, you are in §9 (SI), not §7 (protocol / Completion Timeout).

4.4 Uncorrectable bits (**`UNCOR_STATUS`**, +0x04) — any set after a clean run = FAIL

Bit	Mask	Name	Meaning / typical root cause
4	`0x0000_0010`	Data Link Protocol Error	ACK/sequence-number protocol violation. Protocol/IP bug, sometimes severe SI corrupting seq#s.
5	`0x0000_0020`	Surprise Down	Link dropped to DL_Down unexpectedly. Device/power lost, cable yanked, far-end PHY died, hot-unplug.
12	`0x0000_1000`	Poisoned TLP Received	TLP arrived with the EP (poison) bit set (sender marked its own data bad). Upstream data corruption, not this link's SI.
13	`0x0000_2000`	Flow Control Protocol Error	Credit accounting violated the rules. Firmware/IP bug (or corrupted FC DLLPs).

Bit	Mask	Name	Meaning / typical root cause
14	`0x0000_4000`	Completion Timeout	A non-posted request (usually a read) never got its completion before the CTO timer fired. Upstream hung / wrong address / ASPM-L1 exit too slow / switch dropped it / FW. (sec 7)
15	`0x0000_8000`	Completer Abort	The completer refused the request (returned CA). Target-side: bad address, permission, or completer-internal error.
16	`0x0001_0000`	Unexpected Completion	A completion arrived with no matching outstanding request. Switch mis-routing / duplicate tags / firmware.
17	`0x0002_0000`	Receiver Overflow	More TLP data than advertised credits. Flow-control/IP bug (transmitter overran).
18	`0x0004_0000`	Malformed TLP	Structurally invalid TLP (bad length/byte-enables/addr/type). FW/IP bug or severe corruption. Usually Fatal by default severity.
19	`0x0008_0000`	ECRC Error	End-to-end CRC mismatch (only if ECRC gen+check enabled). End-to-end corruption through a switch/retimer LCRC didn't catch.
20	`0x0010_0000`	Unsupported Request	Target doesn't support the request (access to an unimplemented BAR/region). Address-map / enumeration / driver bug.
21	`0x0020_0000`	ACS Violation	Access Control Services blocked a peer-to-peer TLP. IOMMU/ACS policy (often expected under virtualization).
22	`0x0040_0000`	Uncorrectable Internal Error	Device-internal uncorrectable (its own logic/RAM). Device-side.
23	`0x0080_0000`	MC Blocked TLP	Multicast-blocked TLP.
24	`0x0100_0000`	AtomicOp Egress Blocked	Atomic op blocked at egress.

Bit	Mask	Name	Meaning / typical root cause
25	`0x0200_0000`	TLP Prefix Blocked	TLP prefix egress-blocked.
26-31	–	Poison-egress / DMWr-egress / IDE / PCRC checks	Newest-spec integrity/encryption + translation errors. Decode them so an unexpected high bit isn't shown as "unknown."

Why the high bits matter for a tool. A naive decoder that stops at bit 22 shows an IDE or AtomicOp error as "unknown bit 24," which sends a tech down the wrong path. Decode the full word even though bits 23–31 are rare on an ordinary compute board — being able to *name* an unexpected bit is half of triage.

4.5 Severity and mask — read them before you stress

`UNCOR_SEVERITY` (+0x0C) selects Fatal(1)/Non-Fatal(0) per bit; a Fatal error triggers ERR_FATAL → link reset / DPC. `*_MASK` selects whether the error is *reported* (generates a message/interrupt); masked errors may still latch in status. For deterministic measurement, read severity + mask in the arm step (§3) so you know which bits will trigger DPC or a link reset *under you* mid-stress — otherwise your device vanishes and you blame the wrong thing.

5 Worked AER Header-Log Decode

The 16 bytes at AER `+0x1C` are the first 4 DWORDs of the offending TLP — the one that caused the *first* uncorrectable error (per the First Error Pointer in `ERR_CAP[4:0]`). Decoding it turns "Completion Timeout" into "a 4-byte memory read of address `0x05010000` by requester `00:04.0` with tag 0x0a timed out." That is the difference between a guess and a root cause.

5.1 Where to get the raw DWORDs

```
# From lspci (decoded position, raw hex):
lspci -vvv -s $BDF | grep -A1 'Header Log'
#   ...   HeaderLog: 00000001 00200a03 05010000 00050100

# Or straight from the kernel dmesg AER line (it prints the same 4 DWORDs):
dmesg | grep -A4 'PCIe Bus Error' | grep 'TLP Header'
#   0000:50:00.0:   TLP Header: 00000001 00200a03 05010000 00050100

# Or raw from config space (ECAP_AER+0x1C, four DWORDs):
setpci -s $BDF ECAP_AER+0x1c.L ECAP_AER+0x20.L ECAP_AER+0x24.L ECAP_AER+0x28.L
```

5.2 Decoding the four DWORDs

A TLP header's first DWORD carries Fmt and Type, which tell you read vs write, memory vs config vs completion, and 3-DWORD vs 4-DWORD header (i.e. 32- vs 64-bit address). The thing that trips people up: byte 0 holds Fmt in bits **`[7:5]`** and Type in bits **`[4:0]`**, so you read those two fields out of *one* byte. Worked example with the bytes above (`DW0 = 0x00000001`):

```
DW0 = 0x00000001
  byte0 = 0x00 = 0b000_00000
     [7:5] Fmt  = 0b000 -> 3-DWORD header, NO data payload
     [4:0] Type = 0b00000 -> Memory Read/Write class
     => Fmt=000 + Type=00000 = Memory Read Request, 32-bit address (3DW header, MRd32)
  byte1..3 -> TC[6:4], attrs, TH/TD/EP/AT, Length[9:0].
     Length field [9:0] = 0x001 -> 1 DWORD requested (a 4-byte read)

DW1 = 0x00200a03
  [31:16] Requester ID = 0x0020 -> bus[15:8]=0x00, dev[7:3]=0b00100=4, fn[2:0]=0 (00:04.0)
  [15:8]  Tag          = 0x0a
  [7:0]   Byte enables = 0x03 -> last-DW BE [7:4]=0x0 / first-DW BE [3:0]=0x3

DW2 = 0x05010000  -> Address[31:2] (3DW header => 32-bit address); low 2 bits reserved 0
  => target address ~ 0x05010000

DW3 = 0x00050100  -> for a 3DW Mem request the header is only 3 DWORDs, so this 4th word is
                     the next captured word, not header. For a 4DW (64-bit) header DW2/DW3
                     would be Address[63:32] / Address[31:2].
```

Read it as a sentence: *"A 4-byte memory read of address `0x05010000` by requester `00:04.0` (tag 0x0a) is the TLP that triggered the first uncorrectable error."* Now go look: is `0x05010000` inside a BAR that exists? Is `00:04.0` the device you expect? If the uncorrectable bit set was Completion Timeout, that read never completed — so the *completer* of `0x05010000` is the suspect (hung endpoint, ASPM-L1 exit too slow, or a switch that dropped the completion), not the requester's link SI.

> Practical decode shortcuts. Fmt `[7:5]`: `000`=3DW/no-data, `001`=4DW/no-data, `010`=3DW/with-data, `011`=4DW/with-data, `100`=TLP-prefix. Type `[4:0]`: `00000`=Memory, `00010`=I/O, `00100`/`00101`=Config Type0/1, `01010`=Completion (Cpl/CplD). So byte0 reads off directly: `0x00`=MRd32, `0x20`=MRd64, `0x40`=MWr32, `0x60`=MWr64, `0x0A`=Cpl (no data), `0x4A`=CplD (with data), `0x04`/`0x05`=CfgRd0/CfgRd1.

(Note `0x04` is a *Config* read, not a memory read — the two are one bit apart in Type, so mis-reading byte0 sends you at the wrong device.)

A Completion TLP (`byte0 = 0x0A` or `0x4A`) is laid out differently — its DW1/DW2 carry Completer ID, Completion Status, Byte Count, and Lower Address instead of an address:

```
Completion header log (byte0 = 0x4A -> CplD with data):
DW0  [7:5]Fmt=010 (3DW w/data) [4:0]Type=01010 (Completion); Length = payload DWORDs
DW1  [31:16] Completer ID   [15:13] Cpl Status (000=SC,001=UR,010=CRS,100=CA)
     [12] BCM   [11:0] Byte Count
DW2  [31:16] Requester ID   [15:8] Tag   [6:0] Lower Address
```

So an Unexpected Completion header log tells you *who completed* (Completer ID), *for whom* (Requester ID + Tag), and *what status* — which is exactly the mis-routing / duplicate-tag clue. A Completion Timeout, by contrast, logs the *request* that never got its completion (the MRd example above), so you chase the completer of that address.

6 The Latches You Must Watch — LnkSta LBMS / LABS / DLLLA / LT

A 5 ms poll loop on "is it Gen4 x16 right now" *misses* a link that bounced to Recovery and back between polls. PCIe gives you latched status bits in Link Status (`LnkSta`, at PCIe-cap `+0x12`) that *survive* between your reads — these catch the transients a snapshot misses.

`LnkSta` bit	Name	What it tells you
`[3:0]`	Current Link Speed (`CLS`)	The speed *right now* (1->2.5 … 6->64 GT/s). Compare to `LnkCap` max.
`[9:4]`	Negotiated Link Width (`NLW`)	The width right now. Compare to `LnkCap` max.
11	Link Training (`LT`)	Set *while* the link is retraining (in Recovery). Flicking = the link is bouncing.
13	Data Link Layer Link Active (`DLLLA`)	Drops to 0 when the link goes down (DL_Down). A latched link-down detector.
14	Link Bandwidth Management Status (`LBMS`)	W1C latch: set when the link changed speed/width because *software/hardware* initiated it (a managed retrain).
15	Link Autonomous Bandwidth Status (`LABS`)	W1C latch: set when the link changed speed/width *autonomously* (the hardware decided – usually because it couldn't hold the higher rate: a reliability/SI signal).

```
# Snapshot LnkSta:
setpci -s $BDF CAP_EXP+0x12.W             # the 16-bit Link Status word

# Arm the LBMS/LABS latches (W1C: write 1 to bits 14,15 => 0xC000), then stress, then read:
setpci -s $BDF CAP_EXP+0x12.W=0xc000      # clear LBMS|LABS
#   ... run stress / soak ...
setpci -s $BDF CAP_EXP+0x12.W             # re-read: if bit15(LABS) or bit14(LBMS) set,
                                          # the link renegotiated during the window
```

Why LABS is the gem. `LABS` (bit 15) latching means the hardware autonomously dropped speed or width during your soak — i.e. it could train to Gen4 x16 but couldn't *hold* it. A one-shot "current speed = Gen4 x16" check passes that unit; the `LABS` latch fails it correctly. This is the canonical "trains fine, marginal under load/temperature" catch, and it costs one extra register read.

> **`DLLLA`** requires capability. Whether `DLLLA` (bit 13) is meaningful depends on DLL Link Active Reporting Capable (`LnkCap` bit 20). Check that bit before trusting a `DLLLA` drop as link-down; on a port that can't report it, use Surprise Down (AER) and `dmesg` instead.

6.1 LnkSta2 and Flit-Mode (Gen6)

`LnkSta2` is the 16-bit register at PCIe-cap **`+0x32`**. Its bit you care about today is Flit Mode Status **`[10]`** — set when the link is operating in Gen6 FLIT mode. This matters because in FLIT mode the error model changes: DLLPs are eliminated (ACK/NAK and flow-control move *inside* the FLIT), errors are caught by FEC (correctable symbols) + a strong CRC + replay, and AER even gains a "TLP logged in FLIT mode" bit. An AER-LCRC-retry

BERT *under-measures* a Gen6 link — a true Gen6 BERT must read FEC correctable/uncorrectable counters, not just AER correctables. If `LnkSta2[10]` is set, switch your mental model to FEC-based error accounting.

7 Completion Timeout — The ASPM / L1SS / CTO Story

`Completion Timeout` (UNCOR bit 14) is the uncorrectable bit most often *misdiagnosed as SI* when it is actually power-management latency or a hung completer. A non-posted request (usually a memory read) didn't get its completion before the CTO timer fired. The header log (§5) names *what* read and *who* issued it; this chapter is how to find *why it was slow*.

7.1 The CTO timer lives in Device Control 2

The completion-timeout *timer range* is programmable in Device Control 2 (`DEVCTL2`, at PCIe-cap `+0x28`), Completion Timeout Value field **`[3:0]`**. `DEVCAP2` (`+0x24`) advertises which ranges are supported and whether CTO can be disabled.

```
setpci -s $BDF CAP_EXP+0x28.W       # DEVCTL2: [3:0]=CTO value, [4]=CTO disable
setpci -s $BDF CAP_EXP+0x24.L       # DEVCAP2: supported CTO ranges, CTO-disable-supported
```

Default CTO ranges run from ~50 μs up to ~64 s depending on the field; a too-*short* CTO can manufacture timeouts on a perfectly good but slow completer, and a too-*long* CTO can hang the CPU on a dead one.

7.2 The ASPM / L1 substates connection

The classic field "random hang / occasional CmplTO under no obvious stress" is L1-exit latency: the link went into the L1 (or deeper L1.1/L1.2) power-saving substate, and the time to wake it and bring it back to L0 exceeded the CTO timer, so an in-flight read timed out. Causes: an L1SS exit-latency misconfiguration, a too-aggressive ASPM policy, or a completer whose own wake is slow.

Why L1.2 specifically is the danger. The substates trade wake-up time for power:

State	What's kept alive	Typical exit latency	CTO risk
L0s	full common mode, fast	sub-us to a few us	low
L1	common-mode voltage held; PLL off	a few us to ~tens of us	moderate
L1.1	common mode held, no CLKREQ#	tens of us	moderate
L1.2	common mode and reference dropped	~tens of us to ~100 us (T_POWER_ON + common-mode restore)	highest

L1.2 drops the common-mode voltage and the clock, so exit must restore common mode (the `T_POWER_ON` time programmed in the L1 PM Substates Control 2 register) before the link can even start training back to L0. If the minimum CTO timer ($\approx$50 μs in the default range) is shorter than the advertised+actual L1.2 exit latency plus the completer's own wake, a read issued just as the link napped times out. That is the whole bug, and it is a *latency* bug, not a *wire* bug.

```
# What ASPM/L1SS is enabled, and what exit latencies are advertised?
lspci -vvv -s $BDF | grep -iE 'ASPM|L1SubCtl|L1SubCap|LnkCtl:|Latency'
```

A decoded example to read against:

```
LnkCap: ... ASPM L1, Exit Latency L1 <32us
LnkCtl: ASPM L1 Enabled; ...
L1SubCap: PCI-PM_L1.2+ PCI-PM_L1.1+ ASPM_L1.2+ ASPM_L1.1+ L1_PM_Substates+
          PortCommonModeRestoreTime=40us PortTPowerOnTime=10us
L1SubCtl1: PCI-PM_L1.2+ PCI-PM_L1.1- ASPM_L1.2+ ASPM_L1.1-
           T_CommonMode=40us LTR1.2_Threshold=101376ns
L1SubCtl2: T_PwrOn=10us
```

Read it as: *ASPM L1.2 is enabled both ends; restoring common mode takes 40 µs and `T_POWER_ON` is 10 µs, so a worst-case exit is ~50 µs — right at the CTO floor.* If `Exit Latency L1` advertised by the device exceeds what the requester budgeted (the Acceptable Latency in its own DEVCAP), the kernel may even refuse to enable ASPM; when it enables it anyway and you still see CmplTO, the advertisement is optimistic versus silicon reality.

7.3 The decisive A/B test

```
# A: reproduce the CmplTO with ASPM as-shipped (record rate + header log + First Err Ptr).
# B: disable ASPM and re-run the EXACT same stress. Three scopes, coarse -> fine:

# B1 (whole machine, needs reboot) -- the cleanest proof:
#     add to kernel cmdline and reboot:  pcie_aspm=off

# B2 (runtime, all ports, no reboot):
echo performance | sudo tee /sys/module/pcie_aspm/parameters/policy

# B3 (runtime, JUST this link's L1SS, leaves the rest of the box alone):
#     clear ONLY ASPM Control [1:0] in LnkCtl (+0x10) with a masked write so you don't
#     disturb RCB/CommonClk/etc. setpci masked write: VALUE:MASK on the .B (byte) form:
sudo setpci -s $BDF CAP_EXP+0x10.B=0x00:0x03   # ASPM Ctl [1:0] -> 00 (disabled) on the EP
# (do the same on the port above it; L1 needs BOTH ends to agree)
```

If the Completion Timeouts vanish with ASPM off, the root cause is L1-exit latency, not link SI — the fix is an ASPM/L1SS policy, a corrected `T_POWER_ON` / common-mode-restore-time advertisement, or a longer CTO range, and you should *not* be margining lanes or reseating connectors. If they persist with ASPM off, the completer is genuinely hung or mis-addressed (back to the header log and the upstream device). This one A/B test routinely saves a day of chasing the wrong layer.

> Pass/fail call on the line. If B fixes it, the unit is *good hardware with a bad latency config* — fail it to firmware/BIOS, not to the SI/rework bin. If B doesn't fix it, it's a real protocol/completer fault — fail it to the upstream device named by the header log. Recording *which branch* is what makes the call defensible.
>
> Why this is in a PCIe playbook and not a power-management note: because CmplTO *looks* like a link error in AER and in `dmesg`, and the instinct is to suspect the wire. The disciplined move is to rule out ASPM/L1SS *first* with the A/B test, since it's a 10-minute reboot, before spending an hour on margining and reseating.

8 LTSSM-Stuck and Enumeration Failures

If `lspci -nn` doesn't show the device (or shows it with no BARs), the link never reached L0 — the LTSSM is stuck early. You can't read AER on a link that never came up, so this branch is about the *physical bring-up* facts.

8.1 What "stuck" means per LTSSM state

Stuck in	What it means	First moves
Detect	No link partner sensed (RX termination not seen on the far end).	Far-end power off? PERST# not released? AC-coupling cap or termination missing on a lane? Dead PHY? The classic "device not detected."
Polling	Can't get bit/symbol lock from TS1/TS2 (always at 2.5 GT/s here).	REFCLK absent/wrong, SSC mismatch, RX DC offset, gross SI (impedance/crosstalk), polarity inverted.
Configuration	Can't agree width / lane numbers.	Dead or noisy lanes (configures a smaller width), lane-reversal unsupported, bifurcation mismatch, scrambler problem. A link that comes up x8 instead of x16 fell out here on the high lanes.
Recovery (looping)	Comes up then keeps re-entering Recovery.	Marginal SI / EQ preset mismatch / RX won't lock at the new speed / thermal drift. The #1 *dynamic* failure (sec 9).

8.2 The enumeration-failure checklist

```
# 1. Is it there at all?
lspci -nn | grep -i <vendorID>           # right Vendor:Device at the expected BDF?
dmesg | grep -iE 'link (training|up|down)|not ready|timed out|Bifurcation'

# 2. If present but dead to software, are BARs assigned?
lspci -vvv -s $BDF | grep -iE 'Region|BAR|ignoring'   # "can't assign" / "ignoring BAR" = dead

# 3. Rails + REFCLK + PERST#  (this is where the scope comes in, sec 10):
#    - scope the PHY supply rails at power-on (sequencing + level)
#    - confirm REFCLK present at the slot (100 MHz, SSC if expected)
#    - confirm PERST# is released (deasserted) at the right time after rails are up

# 4. Bifurcation vs the schematic (top cause of a custom card not enumerating):
#    Does the BIOS/strap split (e.g. 2x8 or 4x4) MATCH the board's lane assignment?
```

8.3 What these look like in dmesg

The kernel narrates bring-up; learn to read the common lines. (Exact wording varies by kernel version and controller driver — match on the keyword, not the phrasing.)

```
# Link never trains (stuck Detect/Polling) -- the most common "device not detected":
pcieport 0000:00:1c.0: cannot train link: failed to enter L0 (LTSSM ...)
nvme 0000:03:00.0: Device not ready; aborting initialisation
```

```
# Trained but DOWN-NEGOTIATED in speed/width (EQ/SI/gen-cap -- go to sec 9).
# pcie_print_link_status() emits this when CURRENT bandwidth < what the device is CAPABLE of:
pci 0000:03:00.0: 31.504 Gb/s available PCIe bandwidth, limited by 8.0 GT/s PCIe x4 link at
    0000:00:1c.0 (capable of 252.048 Gb/s with 16.0 GT/s PCIe x16 link)
# (a "you're leaving bandwidth on the table" warning -- grep dmesg for 'limited by')

# Link keeps bouncing (Recovery loop -- marginal SI under load/temp, sec 9):
pcieport 0000:00:1c.0: BWMgmt event, current link speed downgraded; ...
pcieport 0000:00:1c.0: device [....] error status/mask=00001000/00002000  (replay-timer correctables)

# Surprise Down / hot-unplug containment (sec 10/11):
pcieport 0000:00:1c.0: DPC: containment event, status:0x1f08 source:0x0000
pcieport 0000:00:1c.0: DPC: unmasked uncorrectable error detected
nvme nvme0: frozen state error detected, reset controller
```

Two reads that save time: the kernel's "available PCIe bandwidth, limited by ..." line is a *free* speed/width-downgrade detector at every boot (grep dmesg for `limited by`), and a DPC containment line tells you the device didn't just disappear — a port *deliberately* killed the link on a fatal error (§11), which is a very different finding from a power glitch.

> Bifurcation is the #1 custom-card enumeration bug. A root-port x16 controller split into 2×x8 or 4×x4 is configured in BIOS/strap and must match the board layout. A mismatch shows up as "device not detected" or "trains at the wrong width." On a Zoox custom card, verify the BIOS bifurcation setting against the schematic's lane assignment *before* you suspect a dead PHY.

9 Signal Integrity — Confirming It and Localizing the Lane

Once you've decoded a correctable cluster (RxErr / BadTLP / BadDLLP / ReplayTO / RollOver, §4) or a speed/width fallback (§2), you suspect SI. This chapter *confirms* SI and localizes it to a lane and a cause — the discriminators are temperature dependence, eye margin, and reseat/cable sensitivity.

9.1 Equalization context (why Gen3+ links fall back)

At 8 GT/s and above the channel attenuates the signal until the raw eye is closed; equalization reopens it (TX FIR: pre-cursor C_{-1}, cursor C_0, post-cursor C_{+1}; RX: CTLE + DFE). The 4-phase EQ handshake runs in Recovery.Equalization; if it can't reach a usable eye at the higher rate it falls back to a lower gen. Causes of fallback: trace length/loss, via stubs, impedance discontinuities, connector seating, temperature (equalizes at 25 °C, fails at 85 °C), power-supply noise, a too-aggressive/too-weak TX preset, or a BIOS `Target Link Speed` cap (`LnkCtl2.TLS`). Because the RX adapts (CTLE/DFE), a marginal link can *still train* — which is exactly why lane margining matters more than pass/fail link-up.

9.2 Retrain and replay analysis — counting the recoveries

A marginal link rarely fails cleanly; it *recovers*, repeatedly, and each recovery is a window where traffic stalls. The two observable rates that quantify this:

- Retrain / recovery entries — every trip through Recovery (to re-equalize or re-lock). You catch these via the `LnkSta.LT` bit toggling and the `LABS` latch (§6), and on many controllers via a hardware recovery-entry counter in a vendor PMU (e.g. the DesignWare / HiSilicon PCIe PMU exposed through Linux `perf`). A rising recovery rate under load/temperature is a marginal-SI fingerprint even when the *final* state reads Gen4 x16.
- Replays — every `Replay Timer Timeout` (COR bit 12) or `REPLAY_NUM Rollover` (COR bit 8) is one or more TLPs that had to be re-sent because the ACK didn't arrive in time. The replay *rate* (correctables per second, deltaed from `aer_dev_correctable`) is your link's "BER proxy" without a BERT: it rises monotonically as the eye closes.

```
# Watch retrain/replay accumulate over a soak window (delta the kernel counters):
watch -n5 'cat /sys/bus/pci/devices/'"$BDF"'/aer_dev_correctable'
# and watch LT flicker + LABS latch (see sec 6) at the same time:
watch -n1 'setpci -s '"$BDF"' CAP_EXP+0x12.W'   # bit11=LT toggling, bit15=LABS latched

# If a vendor PCIe PMU is present, recovery entries show up under perf:
perf list | grep -iE 'pcie|recovery|l0s|l1'      # then perf stat -e <event> ...
```

Decode pattern. *Replays climbing with no retrains* = the link is correcting at the DLL layer (TLPs corrupted, NAK'd, re-sent) but holding L0 — early/moderate SI. *Retrains climbing too* = the eye closed far enough that the RX loses lock and must re-equalize — worse SI, and the source of latency spikes a CmplTO can ride (§7). *Both flat but speed downgraded once* = a single EQ failure at bring-up, not an ongoing-margin problem (suspect a fixed channel/preset issue — try the preset move below — not temperature).

9.3 Reading and overriding the negotiated presets

The Secondary PCIe extended capability (ID `0x0019`) holds the Gen3+ Lane Equalization Control — per lane, the DSP TX preset, USP TX preset, and RX preset hints. Reading it tells you which preset each lane negotiated; on a board where one lane is marginal, a different preset there is a clue.

```
lspci -vvv -s $BDF | grep -A8 'Secondary PCI Express'   # per-lane EQ presets (if decoded)
# Forcing a gen to test EQ at a specific rate: set Target Link Speed then retrain.
# Use MASKED writes so you change only the field you mean (data:mask read-modify-write):
setpci -s $BDF CAP_EXP+0x30.W                  # read LnkCtl2 ([3:0]=Target Link Speed)
setpci -s $BDF CAP_EXP+0x30.W=0x0003:0x000f    # cap Target Link Speed at Gen3 (3), other bits kept
setpci -s $BDF CAP_EXP+0x10.W=0x0020:0x0020    # set ONLY Retrain Link (LnkCtl bit5) to retrain
```

Retrain warning. Writing Retrain Link or Target Link Speed *perturbs a live link* — it will blip the device. The masked (`data:mask`) form above changes only the intended bits, but a retrain still drops and re-trains the link. Do it on a dev station or with the DUT quiesced, never on a unit mid-soak on a production line. Retraining on the DSP side (root/switch downstream port) is the conventional side to drive.

9.4 Lane margining — the scope-free eye, and the real tool

Lane Margining at the Receiver (extended cap ID `0x0027`) is mandatory at $\geq$16 GT/s (Gen4+). It commands a receiver, *while the link stays in L0*, to shift its sampling point in time (left/right) and (if supported) voltage (up/down), per lane, and report when errors appear. Stepping the offset outward until errors exceed a limit measures the eye margin on-die, with no oscilloscope. Timing margining is required at Gen4; voltage margining is mandatory at Gen5 (32 GT/s) and up.

The real Linux tool is **`pcilmr`** — part of `pciutils` ($\geq$ 3.13, May 2024; improved in 3.14). It is a standard tool, no vendor SDK. *This is the answer to "how do I margin a lane today."*

```
# Find links that can be margined (negotiated >=16 GT/s):
sudo pcilmr --scan

# Margin one downstream port, all lanes, both timing + voltage:
sudo pcilmr --margin -TV 0000:03:00.0

# Margin specific lanes / receivers, save CSV, custom grade thresholds:
sudo pcilmr -o ./csv 0000:ab:00.0 -r 1,6 -g 1t=20% -g 1v=f,30 \
            0000:52:00.0 -l 0,1,2 -TV

# Margin every ready link in the system, one at a time:
sudo pcilmr --full -o ./csv
```

Key flags: `-e <errlimit>` (default 4), `-d <dwell-sec>` (default 1), `-l <lanes>`, `-r <recv#>` (1 = the port's own RX ... 6 = far-end RX, including retimers 2–5), `-t/-T` (timing), `-v/-V` (voltage), `-g` (grading in %UI or ps), `-c` (capabilities only). Requires root (extended config), the link must be in D0, and ASPM + HW-autonomous features must be disabled during the test (pcilmr does the latter and warns).

Converting steps to UI and mV (what the tool does internally — useful when you parse its CSV or hand-roll a fallback):

$$\text{timing\_margin\_UI} = \frac{\text{passing\_timing\_steps}}{\text{NumTimingSteps}} \times \frac{\text{MaxTimingOffset}}{100}$$

$$\text{voltage\_margin\_mV} = \frac{\text{passing\_voltage\_steps}}{\text{NumVoltageSteps}} \times \text{MaxVoltageOffset}$$

`MaxTimingOffset` is a percent of a UI (so `/100` gives UI); `MaxVoltageOffset` is in mV (or 10 mV units on some parts — verify per silicon). Spec eye targets `pcilmr` grades against: at 16 GT/s, min timing 30% UI ($\approx$18.75 ps) / recommended 38% UI, min voltage 15 mV / recommended 21 mV; at 32 GT/s, min timing 30% UI ($\approx$9.375 ps), min voltage 15 mV.

Other margining tools (richer reports / programmatic pass-fail): OCP `pci_lmt`, Google `pcie_lmt` (Gen4/5/6), Oxide `lmar`. For a station tool the robust pattern is shell out to **`pcilmr`** and parse its CSV (it already hardcodes vendor quirks like Ice Lake) rather than hand-rolling the register sequence.

9.5 The SI confirmation moves

1. Decode which correctable bit(s) — RxErr/BadTLP/ReplayTO cluster confirms PHY-layer.
2. Margin the lanes (`pcilmr -TV`) — a margin below the limit on one lane localizes it; one weak lane points at a connector pin, a via, a SerDes, or an AC-cap on that lane.
3. Sweep temperature — count correctables hot vs cold; margin shrinking with temperature is the SI fingerprint (and the reason to test hot).
4. Reseat / swap the cable — if the symptom moves with the connector/cable, it's mechanical/contact SI, not silicon.
5. Try a better TX preset — if a preset change improves margin/error rate, the channel was under-equalized.

10 Power and Mechanical — Surprise Down and Load-Correlated Errors

When AER shows Surprise Down (UNCOR bit 5), or correctable errors come in bursts that correlate with load steps, suspect power integrity or a mechanical/contact problem, not the SerDes channel per se.

Symptom	Power/mechanical reading	Confirm with
Surprise Down under load	The far end lost power / browned out, or a connector momentarily opened.	Scope the PHY/device rail at the load step; reseat; swap PSU; check the connector.
Correctable bursts at load steps	Rail droop / ripple at a current transient is corrupting the eye transiently.	Scope rail AC-coupled under the same load profile; correlate burst timestamps to load.
Errors only under combined load (GPU + NVMe + link all busy)	System power budget / shared-rail droop.	Measure the rail with everything loaded; back off one load and see if errors stop.
Device vanishes then re-enumerates	DPC fired on a fatal error (sec 11) OR a power glitch reset it.	`dmesg` for DPC containment vs a clean re-enumerate; rail scope at the event.

The instrument-side technique (see the Study Guide's instruments and measurement material): 4-wire on the rail under load, scope AC-coupled and bandwidth-limited for ripple, and scope several rails on power-up for sequencing. The through-line worth repeating: a lot of "digital" PCIe link failures are power/SI failures — the engineer who reaches for the scope and the AER decode together closes the intermittent ones.

> The discriminator vs thermal SI: power droop is load-correlated (it tracks current transients and improves when you reduce load even at the same temperature); thermal SI is temperature-correlated (it tracks junction temperature and improves when you cool the part even at the same load). Vary load at fixed temperature, then temperature at fixed load, to separate them.

11 DPC — When the Device Vanishes On Purpose

Downstream Port Containment (DPC, extended cap ID `0x001D`) is a root/switch downstream-port mechanism that, on a Fatal or Non-Fatal error, automatically disables the link to contain the error — it stops bad TLPs from propagating and stops a hung read from hanging the CPU. When DPC fires, the device disappears, and the kernel then attempts recovery (DPC + ERR recovery → re-enumerate).

For manufacturing test DPC is a double-edged sword: it cleanly *captures and isolates* a fatal event (excellent evidence) but it also *yanks the device out from under your test* mid-stress.

```
# Is DPC present and enabled on the port?
lspci -vvv -s <root-or-switch-DSP-bdf> | grep -A3 'DPC:'
#   DPC: Enabled ... Trigger:+ ...   (look for Enabled / Trigger reason)

# After a link-down event, read WHY DPC fired (the trigger reason is the evidence):
#   DPC_STATUS trigger reason: uncorrectable / ERR_NONFATAL / ERR_FATAL / RP-PIO / SW-trigger
#   RP_PIO_* logs the offending root-port read that tripped RP-PIO.
dmesg | grep -iE 'DPC|containment'
```

Decide per-station whether to leave DPC on or off:

- DPC on = a contained fatal error is captured as a clean event with a trigger reason and (for RP-PIO) the offending read logged — great forensics — but your stress run ends when it fires.
- DPC off (e.g. `pcie_ports=native` to let the OS own it, or firmware-disabled) = the link stays up so you can keep stressing and watching, at the cost of letting a fatal error propagate. Use this when you're deliberately *characterizing* how a marginal link behaves past the first fatal event.

Know which one your root ports are set to *before* a stress run, so "the device vanished" is interpreted correctly (DPC contained a real fatal error vs a power glitch vs a hot-unplug).

> **`pcie_ports=native`** on the kernel cmdline makes the OS own AER/DPC/hotplug even if ACPI `_OSC` didn't grant it — useful when firmware is stingy and you want the kernel's DPC + AER recovery + `aer_dev_*` counters to all be live.

12 Switches, Retimers, and the Secondary-Bus Tree

Zoox builds custom switch/fan-out boards and cabled board-to-board links, so you will rarely debug a single point-to-point link in isolation — you debug a tree. Walk all of it.

12.1 Walk the whole topology, check every link

```
lspci -tv                                  # the tree: root ports -> switches -> endpoints
# For EVERY link in the path (root port, switch USP, each switch DSP, endpoint),
# check speed/width and AER -- not just the endpoint:
for bdf in 0000:00:1c.0 0000:02:00.0 0000:03:00.0 ; do
  echo "== $bdf =="
  cat /sys/bus/pci/devices/$bdf/current_link_speed /sys/bus/pci/devices/$bdf/current_link_width
  setpci -s $bdf ECAP_AER+0x10.L ECAP_AER+0x04.L   # COR / UNCOR status on this link
done
```

A PCIe switch is one Upstream Port + N Downstream Ports internally bridged; each port has its own LTSSM, AER, and link registers. An endpoint's errors may be reported by the switch downstream port above it, or aggregated at the root port — so a fallback or an error cluster can be on *any* segment, and you must check each. The switch is itself a unit-under-test: bad switch SerDes, a switch-firmware flow-control bug (→ FCP / RX_OVER), or a switch that drops completions (→ Completion Timeout at the endpoint above it).

12.2 Retimers appear as extra margining receivers

A retimer is a protocol-aware repeater that recovers the data, re-equalizes, and re-transmits a clean eye — it resets the jitter/loss budget and creates an independent link segment on each side (PCIe allows up to 2 retimers per link). The manufacturing-test gold: retimers show up in lane margining as additional Receiver Numbers (**`pcilmr -r 2..5`**), so you can margin the retimer's RX and localize a marginal segment to "before vs after the retimer." On a cabled link, that tells you whether the bad eye is on the board trace or on the cable.

```
# Margin the near RX (recv#1), the retimer RXs (2..5 if present), and the far RX (recv#6):
sudo pcilmr --margin -TV -r 1,2,3,6 0000:03:00.0
```

A redriver, by contrast, is an *analog* repeater — invisible to software, no link state of its own, you cannot margin or query it. If a link has a redriver and a marginal eye, you're back to the scope.

12.3 Error Source ID localizes the requester

On the root port (only — §4), `Error Source ID` (+0x34) gives the requester ID of the correctable and uncorrectable source. On a tree with switches this is how you attribute an error to the true originating device rather than the port that happened to log it.

13 Self-Testing the AER Pipeline with `aer-inject`

Before you trust a station's AER decode/clear/count path, validate it with no bad hardware by *injecting* a known error and asserting the pipeline reports exactly that. This also makes a strong bring-up artifact: "my tool's AER path is self-tested on a known input."

Requires a kernel built with `CONFIG_PCIEAER_INJECT` (creates `/dev/aer_inject`).

```
# aer-inject config-file grammar (jderrick/intel aer-inject):
cat > badtlp.aer <<'EOF'
AER
PCI_ID 0000:03:00.0
COR_STATUS BAD_TLP
HEADER_LOG 0x04000001 0x00200a03 0x05010000 0x00050100
EOF

sudo aer-inject badtlp.aer
dmesg | tail        # confirm the injected BadTLP appears AND decodes correctly,
                    # and that your tool's clear/read/count path counts exactly one.
```

Accepted COR symbols: `RCVR`, `BAD_TLP`, `BAD_DLLP`, `REP_ROLL`, `REP_TIMER`. Accepted UNCOR symbols: `TRAIN`, `DLP`, `POISON_TLP`, `FCP`, `COMP_TIME`, `COMP_ABORT`, `UNX_COMP`, `RX_OVER`, `MALF_TLP`, `ECRC`, `UNSUP`. Inject one of each and assert: (a) the correct bit is set in status, (b) the header log matches what you injected, (c) the First Error Pointer points at the right uncorrectable bit, (d) your W1C clear actually clears it, (e) your kernel-counter delta is exactly one. That five-point assertion *is* the AER self-test.

> Why this matters on the line. A station whose AER decode is subtly wrong (off-by-one bit, doesn't read the header log, races the kernel) will mis-triage every real failure. `aer-inject` lets you prove the pipeline is correct on a *known* input, so when it fires on *unknown* hardware you trust it. It's the AER analog of a golden-unit correlation.

14 The Symptom → Evidence → Cause Master Table

This is the playbook on one page. Read left to right: the symptom you observe, the evidence that *confirms* a specific cause (not just consistent with it), the root-cause class, and the first fix to try.

Symptom (what you see)	Confirming evidence	Root-cause class	First moves
Device not detected (`lspci` empty)	`dmesg` LTSSM stuck in Detect/Polling; rails/REFCLK/PERST# wrong on scope; BIOS bifurcation != schematic	Bring-up / power / bifurcation	Scope rails+REFCLK+PERST#; verify bifurcation vs schematic; reseat
Detected but no BARs / driver won't bind	`lspci -vvv` "can't assign / ignoring BAR"; `lspci -k` no driver; device in D3	Enumeration / address-map / driver	Check BAR assignment, kernel cmdline, driver blacklist, power state
Trained below max speed (Gen4->Gen3)	`dmesg` gen-change; persists cold AND hot? vs hot-only; `pcilmr` margin shrinks; LnkCap ceilings agree	Equalization / SI margin (or BIOS gen-cap, or thermal)	Compare LnkCap both ends; retest hot+cold; margin lanes; check `LnkCtl2.TLS` cap
Trained below max width (x16->x8)	per-lane `pcilmr` finds a dead/weak lane; reseat changes it; BIOS bifurcation	Connector / dead lane / bifurcation / lane-reversal	Reseat; per-lane margin; check bifurcation & lane-reversal
Correctable cluster (RxErr/BadTLP/ReplayTO/RolloverErr) rising	count rises with temperature; margin low on one lane; reseat/cable swap changes it; better preset helps	Signal integrity (PHY)	Decode bits; margin hot; reseat; try preset; scope ripple
Completion Timeout (UNCOR 14)	header log names the read+requester; vanishes with **`pcie_aspm=off`** -> L1; persists -> completer	ASPM/L1 latency OR hung/mis-addressed completer	Run the `pcie_aspm=off` A/B; decode header log; check upstream device/switch
Malformed / Unexpected Completion / FCP / RxOverflow	header log names the TLP; temperature-independent; tied to a traffic pattern or FW/driver version; switch in path	Protocol / firmware / IP bug	Decode header log; check switch FW; bisect driver/FW version; check credits
Poisoned TLP / ECRC	header log points upstream; ECRC only if enabled end-to-end	Upstream data corruption (through switch/retimer)	Trace upstream; check the data source's ECC; enable ECRC to localize
Surprise Down (UNCOR 5)	correlates with load steps / rail droop on scope; reseat; PSU swap changes it	Power / mechanical	Scope rail under load; reseat; swap PSU; check connector
Frequent retrains but final state looks fine	`LnkSta.LT` toggles; **`LABS`** latch set after soak; recovery-entry counters climb	Marginal SI under load/temp	Watch latches over soak; margin; correlate temp/power
Device vanishes then re-enumerates	`dmesg` DPC containment + trigger reason (vs clean re-enum); rail scope at event	DPC fired on a fatal error (or power glitch)	Read DPC trigger reason + RP-PIO log; decide DPC on/off per station

Symptom (what you see)	Confirming evidence	Root-cause class	First moves
Errors only hot	margin-vs-temp curve closes; correctables track junction temp at fixed load	Thermal SI	Thermal sweep; margin hot; improve cooling/mount
Errors only under combined load at fixed temp	errors track current transients; back off one load -> stop	Power-budget / rail droop	Scope rail loaded; reduce one load; check shared-rail design
Field-flaky, nothing reproduces on the bench	bench passes link-up but one lane's **`pcilmr`** margin is below limit	Latent SI a pass/fail test misses	Add margining to the test; set a data-driven UI limit

The one rule this table encodes: *never report* `errors=N`. Report the tuple — which bits, which lane, the header log, the retrain/LABS state, the margin, and the temperature/load dependence — because that tuple is what selects a row in this table.

15 Command Quick-Reference

Everything in one place for the bench. `BDF=0000:03:00.0` throughout.

Enumerate & topology

```
lspci -nn                       # vendor:device IDs at each BDF
lspci -tv                       # tree: switches, retimers, what's behind what
lspci -nnk -s $BDF              # driver bound (-k) + IDs for one device
lspci -vvv -s $BDF              # FULL: LnkCap/LnkSta, AER status+header log, DPC, margining
```

Speed/width (sysfs, no root)

```
cat /sys/bus/pci/devices/$BDF/current_link_speed   /sys/bus/pci/devices/$BDF/max_link_speed
cat /sys/bus/pci/devices/$BDF/current_link_width   /sys/bus/pci/devices/$BDF/max_link_width
# Whole-machine degraded-link scan:
for d in /sys/bus/pci/devices/*; do
  cur=$(cat $d/current_link_speed 2>/dev/null); max=$(cat $d/max_link_speed 2>/dev/null)
  [ -n "$cur" ] && [ "$cur" != "$max" ] && echo "$(basename $d): $cur (max $max)"
done
```

AER via setpci (ECAP_AER alias)

```
setpci -s $BDF ECAP_AER+0x04.L                 # UNCOR status (read)
setpci -s $BDF ECAP_AER+0x10.L                 # COR status (read)
setpci -s $BDF ECAP_AER+0x0c.L                 # UNCOR severity
setpci -s $BDF ECAP_AER+0x08.L  ECAP_AER+0x14.L  # UNCOR mask, COR mask
setpci -s $BDF ECAP_AER+0x18.L                 # ERR_CAP (First Error Pointer [4:0])
setpci -s $BDF ECAP_AER+0x1c.L ECAP_AER+0x20.L ECAP_AER+0x24.L ECAP_AER+0x28.L  # header log
setpci -s $BDF ECAP_AER+0x10.L=0xffffffff      # clear COR (W1C)
setpci -s $BDF ECAP_AER+0x04.L=0xffffffff      # clear UNCOR (W1C)
```

Link registers via setpci (CAP_EXP alias = PCIe cap base)

```
setpci -s $BDF CAP_EXP+0x0c.L                  # LnkCap (max speed/width, DLLLARC bit20)
setpci -s $BDF CAP_EXP+0x12.W                  # LnkSta (CLS/NLW/LT/DLLLA/LBMS/LABS)
setpci -s $BDF CAP_EXP+0x12.W=0xc000           # arm LBMS|LABS latches (W1C)
setpci -s $BDF CAP_EXP+0x32.W                  # LnkSta2 (Flit Mode Status [10])
setpci -s $BDF CAP_EXP+0x24.L  CAP_EXP+0x28.W  # DEVCAP2, DEVCTL2 (CTO value [3:0])
setpci -s $BDF CAP_EXP+0x10.W=0x0020:0x0020    # Retrain Link (LnkCtl bit5), masked [perturbs link!]
```

Kernel-side

```
dmesg | grep -iE 'pcie|aer|link|train|dpc|bifurcation'
cat /sys/bus/pci/devices/$BDF/aer_dev_correctable   # kernel-maintained counters
cat /sys/bus/pci/devices/$BDF/aer_dev_nonfatal      /sys/bus/pci/devices/$BDF/aer_dev_fatal
# Kernel cmdline knobs: pcie_ports=native (OS owns AER/DPC), pci=noaer (A/B), pcie_aspm=off
```

Margining

```
sudo pcilmr --scan                             # links that can be margined (>=16 GT/s)
sudo pcilmr --margin -TV $BDF                  # all lanes, timing+voltage
sudo pcilmr --margin -TV -r 1,2,3,6 $BDF       # near RX, retimer RXs, far RX
sudo pcilmr -o ./csv --full                    # every ready link, CSV out
```

Self-test

```
sudo aer-inject badtlp.aer                     # inject a known error (needs CONFIG_PCIEAER_INJECT)
```

16 Accuracy Notes & Caveats

This playbook prefers primary sources (kernel `pci_regs.h`, the kernel AER HOWTO, the `pcilmr` man page, PCI-SIG/vendor material). A few things are version- or silicon-dependent and you should confirm against *your* hardware before driving it:

- Lane Margining control-register bit offsets. The *command model* (Receiver Number, Margin Type, Margin Payload, the query→set-limit→step→dwell→read protocol) is well-confirmed; the exact *bit positions* of those fields are the conventional layout — confirm against the PCIe Base Spec revision for your silicon. In practice, drive **`pcilmr`** and parse its CSV rather than hand-coding the register sequence; it hardcodes the per-vendor quirks.
- **`MaxVoltageOffset`** units — mV vs 10 mV on some parts. Verify per silicon before converting steps to mV.
- Gen6 FLIT byte split and the FEC error model — the precise FLIT layout is spec-final; the key operational point (FEC-corrected symbols + CRC + replay replace AER-LCRC-retry accounting, and DLLPs are gone) is what changes your measurement.
- TX preset dB rounding / P10 definition — the coefficient *ratios* are authoritative; some references round the dB columns differently.
- Register offsets you write — always validate against `lspci -vvv` for the specific device before a `setpci` write; a wrong offset on a live link is how you turn a diagnosis into an outage.

Sources (consolidated): Linux kernel `include/uapi/linux/pci_regs.h` (AER, LnkCap/Ctl/Sta, DPC, DEVCTL2 defines); kernel docs — PCIe AER HOWTO, sysfs-pci, DWC/HiSilicon PCIe PMU; `pciutils` — `pcilmr(8)` man page and ChangeLog; lane-margining tools — OCP `pci_lmt`, `google/pcie_lmt`, `oxide-computer/lmar`; `aer-inject` SPEC and kernel `aer_inject.c`; LTSSM & flow control — Shane Colton PCIe Deep Dive Pt.4/Pt.5; equalization — Teledyne LeCroy, Intel Gen3 EQ, bitsilica, the PCIe 3.0 preset table; Gen6/PAM4/FLIT/FEC — VIAVI, Synopsys.